Project 6 (Bonus) — LangChainPro-Branching-Parallel-And-Retry-Patterns-For-A-Real-Customer-Support-Pipeline

Title: LangChainPro Branching Parallel And Retry Patterns For A Real Customer Support Pipeline

Type: Single LCEL pipeline (no agent loop, no LangGraph) · **Stack:** Python, LangChain (LCEL), `langchain-openai`, Pydantic · **Level:** Intermediate

Tagline: A customer-support ticket pipeline built only from LCEL pieces — a structured classifier, a reply chain per category, parallel lookups, a safe fallback and a live-streamed reply — proving plain LangChain is enough for a fixed multi-step job.

> This is the **reading guide** — the story, the folder structure and why each part exists, setup, the 4 build steps, and which document to read for each idea. The other file, README.md, is the **repo README** — you copy it into your repo folder at the end and push it to GitHub. You write every folder, file and line of code yourself; the only thing already created for you is the empty repo folder `LangChainPro-Branching-Parallel-And-Retry-Patterns-For-A-Real-Customer-Support-Pipeline/` .

Charter (what this project is)

A terminal pipeline that works through a queue of support tickets: classify each one, send it to the right reply chain, look up the customer's data in parallel, survive a failing model call, stream the reply, and end with one checked record per ticket. Everything is Doc05's LCEL — `prompt | model | parser` — and nothing else. No agent loop, no graph. This project proves you can apply Doc05 to a real job, and that you know where plain LangChain stops being enough.

Jump to: Read first · Project structure · Setup · Step 1 · Step 2 · Step 3 · Step 4 · Run and test · Checklist · Publish

The Story — what you're actually building

A support ticket lands in a queue: "I was charged twice for my Pro plan this month." Before any person reads it, a lot can already happen. First, decide what kind of ticket it is — billing, technical or general — so it goes to the right kind of reply. Second, for billing and technical tickets, pull up the customer's account and recent orders — both at once, since neither lookup needs the other. Third,

write the reply in a way that survives a bad minute at the API, instead of crashing the queue. Fourth, show the reply live to the support agent watching, and save one checked record the ticketing system can store — including "a human should look at this", and why.

Each of those is one Doc05 idea, applied: a structured-output chain, a plain function as a step, a `dict` of steps that runs in parallel, `.with_fallbacks()`, and `.stream()`. It's a straight line from ticket to record — no step ever goes back, waits for a person, or remembers anything after the run. That's exactly why it doesn't need LangGraph.

> **Before you read further — think about it yourself:** the fourth sample ticket reads half like billing and half like a bug. Should the classifier quietly pick one and move on, or should that doubt be visible at the end? And when the model is down and a double-charge ticket gets a canned "we got your message" reply — is that acceptable, or should it fail loudly? Decide before Step 3.

What you're building, in one line: a support-ticket pipeline made only of LCEL chains — classify, look up, reply, record.

Why this needs to exist: to prove, with working code, that a fixed multi-step job doesn't need a heavier framework just because one exists.

When you'd reach for this at a real job: requests with a small, fixed set of categories that never need to loop, pause or remember across restarts — support queues, alert triage, form routing.

How it works, mechanically: every piece is a `Runnable` joined with `|`. The category picks the reply chain; a `RunnableParallel` runs the lookups; `.with_fallbacks()` wraps each chain; `.stream()` prints the reply; a Pydantic `TicketRecord` holds the result.

Why not plain Python and raw SDK calls? You could — Project 1 did. But then the parallel lookups, the fallbacks and the streaming are all code you write and debug yourself, for every chain. **Why not a `try/except` around every call?** Step 3's Basic version does exactly that, and shows what it costs.

Where This Fits

This is a bonus project, outside the main 5-project arc. It comes right after Doc05, because Doc05's own Build Task is one small chain module — this is where those same pieces do a real job together.

```
Project 1                → raw SDK, Python router (Docs 01-04)
Project 6 (you are here) → LangChain LCEL only, one straight pipeline
                           (Doc05)
```

```
Project 3                → LangGraph: loops, pauses, saved state
                           (Docs 08-10)
```

The line between the two frameworks, said simply: LCEL is enough while the path is fixed — even with one branch point and one parallel step. A graph earns its cost when a step must route **back**, loop until a condition is met, or pause for a person and resume later. None of that happens here. (Look ahead, optional: Doc09 — LangChain vs. LangGraph vs. RAG.)

What to Read First (docs this project uses)

Everything this project needs was taught in Docs 01-05 and Project 1:

You need to understand…	Read this	Used in
venv, `.env`, never committing secrets	Doc01 — `.env` files	Setup
`load_config()`, `get_logger()`	Doc01 Build Task	Step 1
`prompt | model | parser`	Doc05 — your first LCEL chain	Steps 1, 2
`.with_structured_output(Model)`	Doc05 — structured parser	Step 1
Catching `ValidationError` by name	Doc05 — structured parser	Step 1
Extra template keys are ignored	Doc05 — template variables	Step 1
A function that returns a `Runnable`	Doc05 Build Task	All steps
`Literal` fields, "shape during"	Doc04 — Structured output	Step 1
A router that logs its reason	Project 1 — Step 4	Step 1
Functions and dicts as steps, `RunnableParallel`	Doc05 — Real-world (part 2)	Step 2
Reading a JSON file	Doc02 — JSON	Step 1
Which errors are transient	Doc04 — Error types	Step 3
Backoff, one retry layer only	Doc02 — Retrying	Step 3
`.with_fallbacks()`, `RunnableLambda`, `.stream()`	Doc05 — Real-world (part 2)	Steps 3, 4
Streaming with `flush=True`	Doc04 — Streaming	Step 4

Reused code, unchanged except for its folder and import lines: `config.py`, `exceptions.py`, `utils/logger.py` (Doc01) and `utils/prompts.py` (Project 1). `models/llm.py` is Doc05's Build Task rule — the model is chosen in one place — as a small `get_model()`.

Nothing here is new: Doc05's Real-world (part 2) exercise already practiced `RunnableParallel` (Step 2), `.with_fallbacks()` with `RunnableLambda` (Step 3), and `.stream()` on a chain (Step 4). Each step reuses its exercise's shape.

Project Structure (the professional layout)

Same rule as Project 1: only folders with a real job. The new one is `chains/` — Project 1 left it out because it had no LangChain; here every piece of work is a chain.

Names: the repo folder vs. the Python package

- **Repo folder:** `LangChainPro-Branching-Parallel-And-Retry-Patterns-For-A-Real-Customer-Support-Pipeline/` — already created, and the exact name of the GitHub repo.
- **Python package:** `langchainpro` (inside `src/`), imported as `from langchainpro.services.pipeline import handle_ticket`.
- **Files:** `snake_case.py`, one job each; test files start with `test_`.
- **Prompt files:** named after the category they answer (`billing.txt` answers billing tickets) — the code relies on that.

The final layout (after Step 4)

```
<repo folder>/   (LangChainPro-Branching-...-Customer-Support-Pipeline/)
├── README.md               the repo's front page (copied in at the end)
├── LICENSE                 lets others reuse your code (MIT)
├── .gitignore              keeps .env, .venv/, __pycache__/ out of git
├── .env.example            the setting names, no real values
├── .env                    your real key — never committed
├── pyproject.toml          makes src/langchainpro an installable package
├── requirements.txt        the libraries, for pip install -r
├── src/
│   └── langchainpro/
│       ├── __init__.py
│       ├── main.py             runs the ticket queue — the file you run
│       ├── config.py           Config, load_config()        (Doc01)
│       ├── exceptions.py       MissingConfigError           (Doc01)
│       ├── schemas.py          TicketCategory, TicketRecord
│       ├── models/
│       │   ├── __init__.py
│       │   └── llm.py          get_model()
│       ├── chains/
│       │   ├── __init__.py
│       │   ├── classifier.py   classify_ticket() -> TicketCategory
│       │   ├── replies.py      build_reply_chain(category)
│       │   ├── lookups.py      build_lookup_step() (RunnableParallel)
```

```
│       │   └── fallback.py     with_canned_reply(), with_unsure_category()
│       ├── services/
│       │   ├── __init__.py
│       │   └── pipeline.py     handle_ticket() -> TicketRecord
│       ├── prompts/
│       │   ├── classifier.txt  the sorting rules
│       │   ├── billing.txt     the billing reply rules
│       │   ├── technical.txt   the technical reply rules
│       │   └── general.txt     the general reply rules
│       └── utils/
│           ├── __init__.py
│           ├── logger.py       get_logger()                (Doc01)
│           └── prompts.py      load_prompt(name)        (Project 1)
├── tests/
│   ├── unit/
│   │   ├── test_schemas.py     no API key needed
│   │   ├── test_lookups.py     no API key needed
│   │   ├── test_fallback.py    no API key needed
│   │   └── test_review.py      no API key needed
│   └── integration/
│       └── test_live_pipeline.py real API calls
├── scripts/
│   └── time_lookups.py         one after another vs. parallel
└── data/
    └── sample/
        └── tickets.json        the 4 test tickets
```

Why each folder exists

- **`src/langchainpro/`** — the package. `schemas.py` sits at the top because the chains, the service and the tests all use it.
- **`models/`** — builds the chat model. **Why:** every chain gets the same model; changing it is one line.
- **`chains/`** — one file per chain job: classify, reply, look up, fall back. **Why:** each chain can be built and tested alone, and the files mirror the pipeline's steps.
- **`services/`** — the flow: classify → reply → record. **Why:** no `print()` of results or `input()` here beyond the live stream, so `main.py` and the tests call the exact same `handle_ticket()`.
- **`prompts/`** — one `.txt` per chain. **Why:** reword how billing replies sound without touching Python.
- **`utils/`** — the logger and the prompt loader, used everywhere.
- **`tests/unit/`** — schemas, lookups, fallbacks and review rules, with no API key. **Why:** fast and free — run after every change.

- **`tests/integration/`** — the real classifier and pipeline. **Why:** costs money, so it's run on purpose.
- **`scripts/`** — `time_lookups.py`, a measurement you read, not a pass/fail test.
- **`data/sample/`** — the 4 test tickets, shared by `main.py` and the integration test.

Why each root file exists

- `README.md` — what someone sees first on GitHub.
- `LICENSE` — lets others legally reuse your code (GitHub can add MIT).
- `.gitignore` — keeps `.env`, the venv and cache files out of git.
- `.env.example` — which settings to create, with no real values.
- `pyproject.toml` — the package's name, libraries and the `langchainpro` command.
- `requirements.txt` — the library list for `pip install -r requirements.txt`.

Why each file exists (what and why)

Every file you'll write, in one place — the same What/Why list the documents give after each tree. The steps below build these files one at a time; come back here whenever you forget why a file exists.

- `__init__.py` (every code folder)
  - **What:** An empty file. You create one in every folder that holds Python code — nothing goes inside it.
  - **Why:** Python only treats a folder as a package when this file is there. Without it, an import like `from <package>.<folder> import ...` fails with `ModuleNotFoundError`.
- `main.py`
  - **What:** `main()` loads the config, reads `tickets.json`, runs `handle_ticket()` for every ticket, and prints each `TicketRecord` with `print_record()`.
  - **Why:** One starting point that runs the ticket queue and prints the results. The chains live in other files.
- `config.py`
  - **What:** A `Config` dataclass with two fields (`openai_api_key`, `log_level`), a helper `require_env(key)` that raises `MissingConfigError` when a setting is empty, and `load_config()`, which calls `load_dotenv()`, reads both settings and returns a `Config`.
  - **Why:** Every file gets its settings from one place. If the key is missing, the app stops at startup with one clear line — without this file, the missing key would show up later as a confusing API error.
- `exceptions.py`
  - **What:** A few small classes that inherit from `Exception` — at least `MissingConfigError`. Each one is just `class Name(Exception): pass`, with a docstring saying when it's raised.

    - **Why:** Code can catch one specific problem (`except MissingConfigError:`). Without named errors you'd catch every `Exception` , and real bugs would be hidden along with the expected problem.
- `utils/logger.py`
    - **What:** One function, `get_logger(name)` . It gets the logger for that name and, only the first time (`if not logger.handlers:`), adds a console handler with a timestamp format and the level from `LOG_LEVEL` .
    - **Why:** Every file logs in the same format with one line of code. Without the `if not logger.handlers:` guard, every log line would print twice.
- `utils/prompts.py`
    - **What:** One constant, `PROMPTS_DIR` (the prompts folder, from the repo root), and one function, `load_prompt(name)` , which opens `PROMPTS_DIR + "/" + name + ".txt"` and returns its text.
    - **Why:** Prompts live in text files, so you can change how an agent behaves without editing Python — and no agent has to open files itself.
- `schemas.py`
    - **What:** Two Pydantic classes: `TicketCategory` (`category` as `Literal["billing", "technical", "general"]` , `reason` , `sure: bool`) and `TicketRecord` (customer, ticket, category, reply, `used_fallback` , `needs_human_review` , `review_reasons`). The three fields added after Step 1 have defaults, so older code still builds a record.
    - **Why:** The classifier's answer is checked, and every ticket's result has the same fields — easy to print, store or review.
- `models/llm.py`
    - **What:** A `MODEL_NAME` constant and one function, `get_model()` , which returns `ChatOpenAI(model=MODEL_NAME, temperature=0)` .
    - **Why:** Every chain and agent gets the same model and settings from here. To try another model you change one line, not every file.
- `chains/classifier.py`
    - **What:** `build_classifier_chain()` returns `prompt | get_model().with_structured_output(TicketCategory)` , wrapped with `with_unsure_category()` ; `classify_ticket(ticket)` invokes it and returns the `TicketCategory` .
    - **Why:** Each kind of ticket needs a different reply. Sorting first means each reply prompt stays short and focused.
- `chains/replies.py`
    - **What:** `build_reply_chain(category)` : load the prompt file with the category's name, build `lookups | prompt | model | parser` , and wrap it with `with_canned_reply()` .
    - **Why:** Each category gets its own rules, so billing and technical replies each stay on topic.
- `chains/lookups.py`

    - **What:** Fake `FAKE_ACCOUNTS` and `FAKE_ORDERS` data, three small functions that read the inputs (`get_ticket_text` , `get_account_info` , `get_order_history`), and `build_lookup_step()` , which returns a `RunnableParallel` running the lookups at the same time.
    - **Why:** Two lookups that don't depend on each other finish faster together than one after the other.
- `chains/fallback.py`
    - **What:** `TRANSIENT_ERRORS` (connection errors and timeouts, rate limits, server errors), a `FALLBACK_REPLY` text, `canned_reply()` and `unsure_category()` fallback functions, and `with_canned_reply(chain)` / `with_unsure_category(chain)` , which call `chain.with_fallbacks([...], exceptions_to_handle=TRANSIENT_ERRORS)` .
    - **Why:** During an outage the customer still gets a polite reply, while a bad key or a code bug still fails loudly.
- `services/pipeline.py`
    - **What:** `stream_reply(chain, inputs)` prints each chunk from `chain.stream(inputs)` and returns the full text; `review_reasons(category, used_fallback)` returns the list of reasons for a human check; `handle_ticket(ticket, customer_id)` classifies, builds the reply chain, streams the reply, and returns a `TicketRecord` .
    - **Why:** The whole flow is one function that `main.py` and the tests can call, and the reply appears word by word instead of all at once.
- `prompts/*.txt`
    - **What:** Four text files: the classifier's sorting rules, and one reply prompt per category (billing, technical, general).
    - **Why:** You can change the rules for one kind of ticket without touching the code or the other prompts.
- `tests/unit/test_schemas.py`
    - **What:** Three test functions, no API key: `test_good_category()` builds a `TicketCategory` with `billing` , `test_unknown_category()` checks that `shipping` raises `ValidationError` , and `test_record()` builds a `TicketRecord` from the 4 Step 1 fields. Each prints its case line.
    - **Why:** It proves the classifier can never return a category no branch handles, and that old records still build after new fields are added.
- `tests/unit/test_lookups.py`
    - **What:** Three test functions, no API key: a known customer gets account and order data, an unknown customer gets the "not found" texts, and `build_lookup_step()` returns the keys `ticket` , `account` , `orders` in under 1.5 seconds.
    - **Why:** It catches a change that quietly makes the lookups run one after another again.
- `tests/unit/test_fallback.py`
    - **What:** A `ChatOpenAI` pointed at a port where nothing listens, plus a working step and a buggy step. Three test functions check that a real outage gives `FALLBACK_REPLY` , a working

chain never uses it, and a `KeyError` is still raised.
    - **Why:** It proves the fallback fires during an outage and never hides a bug — with no key and no cost.
- `tests/unit/test_review.py`
    - **What:** A helper `make_category(sure)` and three test functions that call `review_reasons()` : a sure ticket gets no reasons, an unsure ticket gets one, and a fallback reply gets one.
    - **Why:** Each review rule is checked alone, so a rule that stops firing is noticed at once.
- `tests/integration/test_live_pipeline.py`
    - **What:** Test functions that run `handle_ticket()` on the sample tickets with the real API, print each category next to the expected one, and check that the unclear ticket is flagged for a human.
    - **Why:** It proves each ticket gets the right category and a real reply.
- `scripts/time_lookups.py`
    - **What:** One `main()` that times the two lookups one after the other, then with the parallel step, and prints both times.
    - **Why:** It shows you, with real numbers, how much time `RunnableParallel` saves.
- `data/sample/tickets.json`
    - **What:** A JSON list of 4 test tickets, each with a `customer_id` , the `ticket` text and the `expected` category.
    - **Why:** The same inputs every run, shared by the app and the tests.

How imports work in this layout

Exactly as in Project 1: an empty `__init__.py` in every folder with Python files (not `prompts/` or `data/`), `pip install -e .` once, imports like `from langchainpro.chains.replies import build_reply_chain` , and **every command run from the repo root** — the prompts folder and `data/sample/tickets.json` are found from there.

Left out on purpose (and when they come)

Folder / file in a big template	Why not here	Where it appears
`graph.py` , `state.py` (LangGraph)	No loops, no pauses, no saved state	Project 3 (Doc09)
`RunnableBranch`	An `if` and a prompt file per category are clearer	Only if branches grow into whole different chains
`.with_retry()`	The OpenAI client already retries; a second layer stacks	If you ever turn the client's retries off

Folder / file in a big template	Why not here	Where it appears
`.batch()`	One ticket at a time is fine for a demo queue	When the queue gets large
`api/`, a real database	A terminal app with stand-in lookups	Project 5 (Doc12)
`Dockerfile`, CI	Nothing to deploy yet	Doc12-13
pytest, `conftest.py`	Plain test scripts, same as Project 1	Doc13
A frontend	Streaming works in the terminal	Streamlit, when a project needs one

How the structure grows, step by step

Step	New files	Changed files
Setup	root files, `src/langchainpro/__init__.py`	—
1	`config.py`, `exceptions.py`, `utils/logger.py`, `utils/prompts.py`, `models/llm.py`, `schemas.py`, `prompts/` (4 files), `chains/classifier.py`, `chains/replies.py`, `services/pipeline.py`, `main.py`, `data/sample/tickets.json`, `tests/unit/test_schemas.py`, `tests/integration/test_live_pipeline.py`	—
2	`chains/lookups.py`, `scripts/time_lookups.py`, `tests/unit/test_lookups.py`	`prompts/billing.txt`, `prompts/technical.txt`, `chains/replies.py`
3	`chains/fallback.py`, `tests/unit/test_fallback.py`	`schemas.py`, `chains/classifier.py`, `chains/replies.py`, `services/pipeline.py`, `main.py`
4	`tests/unit/test_review.py`	`schemas.py`, `services/pipeline.py`, `main.py`, `test_live_pipeline.py`

Setup (do this once, before Step 1)

Read first: Doc01 — Virtual environments, Doc01 — `.env` files. The same steps as Project 1, with a different package name and library list.

1. Go into the repo folder, start git, and make a venv. From the curriculum's root folder (`multi-agent-ai-engineering-curriculum/`):

```
cd project_6_langchainpro_lcel_patterns
# the \ joins the two lines into one folder name
cd LangChainPro-Branching-Parallel-And-Retry-Patterns-For-A-Real-\
Customer-Support-Pipeline
git init
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
```

2. Create `.gitignore` **before** your first commit:

```
# secrets — never pushed
.env
# the virtual environment — rebuilt from requirements.txt
.venv/
# Python's compiled cache files
__pycache__/
*.pyc
# created by "pip install -e ."
*.egg-info/
```

3. Create `.env` and `.env.example` , the same as Project 1:

```
# .env — never commit this file
OPENAI_API_KEY=sk-your-real-key-here
LOG_LEVEL=INFO
```

```
# .env.example — commit this one
# Copy this file to .env and fill in the real values. Never commit .env.
OPENAI_API_KEY=
LOG_LEVEL=INFO
```

4. Create `requirements.txt` :

```
langchain-core>=1.0
langchain-openai>=1.0
pydantic>=2.0
python-dotenv>=1.0
```

New since Project 1: `langchain-core` (prompts, parsers, `RunnableParallel`) and `langchain-openai` (`ChatOpenAI` , which brings `openai` with it) — Doc05's packages topic.

5. Create `pyproject.toml`:

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "langchainpro"
version = "0.1.0"
description = "A support-ticket pipeline built only with LangChain LCEL."
requires-python = ">=3.10"
dependencies = [
    "langchain-core>=1.0",
    "langchain-openai>=1.0",
    "pydantic>=2.0",
    "python-dotenv>=1.0",
]

[project.scripts]
# after "pip install -e .", typing "langchainpro" runs main()
langchainpro = "langchainpro.main:main"

[tool.setuptools.packages.find]
# the package code lives under src/, not in the repo root
where = ["src"]
```

6. Create an empty `src/langchainpro/__init__.py`, then install:

```
pip install -r requirements.txt
pip install -e .
```

Check it worked: `python -c "import langchainpro; print('ok')"` prints `ok`.

7. Create `LICENSE` — the MIT license, so others may reuse your code. Put the current year and your own name on the Copyright line:

```
MIT License

Copyright (c) 2026 Your Name

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
```

```
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

A Real Example (so this isn't just theory)

Scenario: SupportCo is a small SaaS company. Step 1 saves these 4 tickets in `data/sample/tickets.json`:

1. `C-1001`: "I was charged twice for my Pro plan this month (INV-2201)." → **billing**.
2. `C-1002`: "The export button spins forever and never downloads." → **technical**.
3. `C-1003`: "Do you have a referral program for other companies?" → **general** (no lookups needed).
4. `C-1004`: "My invoice and the dashboard numbers don't match. Bug?" → **billing or technical** — the honest answer is "not sure", and the pipeline must show that.

The lookups (Step 2) are stand-ins for a real database: `get_account_info()` returns a plan and a signup date, `get_order_history()` returns recent invoices. Each waits 1 second on purpose, like a real network call.

Why This Project Is Good for Your Portfolio

- **What problem it solves:** a real multi-step pipeline — routing, concurrency, failure handling, streaming and a checked record — in plain LangChain.
- **Why it matters:** knowing when you *don't* need a heavier framework is as valuable as knowing the framework.
- **When you'd build something like this at a job:** any ticket, alert or form pipeline with a few fixed categories.
- **How it's built:** a structured classifier, a reply chain per category, `RunnableParallel` lookups, `.with_fallbacks()` on transient errors only, `.stream()`, and a Pydantic record with review reasons.

Problems you'll likely run into, and how to fix them:

Problem	Fix
The unclear ticket silently becomes "billing" and nobody notices	Keep the classifier's `sure` flag and turn it into a review reason (Step 4)
The parallel lookups take as long as sequential ones	Check the two functions don't call each other, and measure with `time_lookups.py`
`KeyError: 'account'` in the billing prompt	The step before the prompt must return a dict with exactly the prompt's keys
The fallback hides a bad key or a bug	Pass `exceptions_to_handle=` — only transient errors fall back
A failed call takes far longer than expected	Two retry layers are stacked — keep only the client's retries
The streamed reply and the record disagree	Build the record from the same joined reply, never from a second call
`ModuleNotFoundError: No module named 'langchainpro'`	The venv isn't active, or you skipped `pip install -e .`

Built During These Documents

05_langchain_fundamentals (on top of Docs 01-04 and Project 1)

Plan Before You Code

See 15_five_projects_index: write down Problem → Requirements → Architecture → Components → Data Flow → Implementation Plan → Coding Tasks. Before coding, write one sentence for what each step proves — if you can't, you'll build all four patterns at once and never know which one does the work.

How To Build This — Step by Step

The project grows step by step, and **after every step it runs**: Step 1 classifies and replies, Step 2 adds parallel lookups, Step 3 survives a failing model, Step 4 streams and records. Build it yourself first; use the hints if you're stuck, and the solution only to compare.

Every command runs from the **repo root** with the venv active.

Step 1 — A Classifier That Sends Each Ticket to the Right Reply Chain

Project: LangChainPro-Branching-Parallel-And-Retry-Patterns-For-A-Real-Customer-Support-Pipeline — Step 1 of 4

- **Where you are now:** Setup is done — nothing runs yet.
- **What's new in this step:** the shared files (config, errors, logger, prompt loader, model), `TicketCategory` and `TicketRecord`, a classifier chain, a reply chain per category, `handle_ticket()`, the sample tickets, `main.py`, and the first tests.
- **What stays the same:** — (first step).
- **Why:** every later step makes one part stronger. Prove the basic shape first — one chain decides, a second chain, picked by that decision, replies.
- **How:** `classify_ticket()` (`prompt | model.with_structured_output(TicketCategory)`) → `build_reply_chain(category)` (the category names the prompt file) → `.invoke()` → a `TicketRecord` → `print_record()`.
- **Working after this step:** `python -m langchainpro.main` prints a category and a reply for each of the 4 tickets.

Read first: Doc05 — your first LCEL chain, Doc05 — structured parser, Doc05 — parser failure, Doc01 Build Task.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Bring in the shared files: `config.py`, `exceptions.py`, `utils/logger.py`, `utils/prompts.py` (folder `src/langchainpro/prompts`), and `models/llm.py` with a no-argument `get_model()`.
2. Write `schemas.py`: `TicketCategory` (`category` as a `Literal` of the 3 names, `reason`, `sure`) and `TicketRecord` (`customer_id`, `ticket`, `category`, `reply`).
3. Write `prompts/classifier.txt` and one reply prompt per category: `billing.txt`, `technical.txt`, `general.txt`.
4. Write `chains/classifier.py` (`build_classifier_chain()`, `classify_ticket()` — catch `ValidationError`, fall back to "general, not sure", log the decision) and `chains/replies.py` (`build_reply_chain(category)`).
5. Write `services/pipeline.py` (`handle_ticket()` prints the reply and returns a `TicketRecord`), `data/sample/tickets.json`, and `main.py` (`print_record()`; loop over the samples; catch `AuthenticationError` and `RateLimitError`).
6. Write `tests/unit/test_schemas.py` and `tests/integration/test_live_pipeline.py`. Run everything, and look closely at ticket 4.

The root files — `.gitignore`, `.env`, `.env.example`, `requirements.txt`, `pyproject.toml` and `LICENSE` — come from Setup; this step adds the files below.

Your files after Step 1:

```
src/langchainpro/
├── __init__.py
├── main.py              (new) runs the 4 sample tickets
├── config.py            (new)
├── exceptions.py        (new)
├── schemas.py           (new) TicketCategory, TicketRecord
├── models/
│   ├── __init__.py      (new)
│   └── llm.py           (new) get_model()
├── chains/
│   ├── __init__.py      (new)
│   ├── classifier.py    (new) classify_ticket()
│   └── replies.py       (new) build_reply_chain(category)
├── services/
│   ├── __init__.py      (new)
│   └── pipeline.py      (new) handle_ticket() -> TicketRecord
├── prompts/
│   ├── classifier.txt   (new)
│   ├── billing.txt      (new)
│   ├── technical.txt    (new)
│   └── general.txt      (new)
└── utils/
    ├── __init__.py      (new)
    ├── logger.py        (new)
    └── prompts.py       (new)
tests/
├── unit/test_schemas.py              (new) no API key needed
└── integration/test_live_pipeline.py (new) real API calls
data/sample/tickets.json              (new) the 4 test tickets
```

Why these files (what and why):

- `main.py`
  - **What:** `main()` loads the config, reads `tickets.json`, runs `handle_ticket()` for every ticket, and prints each `TicketRecord` with `print_record()`.
  - **Why:** One starting point that runs the ticket queue and prints the results. The chains live in other files.
- `config.py`
  - **What:** A `Config` dataclass with two fields (`openai_api_key`, `log_level`), a helper `require_env(key)` that raises `MissingConfigError` when a setting is empty, and `load_config()`, which calls `load_dotenv()`, reads both settings and returns a `Config`.
  - **Why:** Every file gets its settings from one place. If the key is missing, the app stops at startup with one clear line — without this file, the missing key would show up later as a confusing API error.

- `exceptions.py`
  - **What:** A few small classes that inherit from `Exception` — at least `MissingConfigError`. Each one is just `class Name(Exception): pass`, with a docstring saying when it's raised.
  - **Why:** Code can catch one specific problem (`except MissingConfigError:`). Without named errors you'd catch every `Exception`, and real bugs would be hidden along with the expected problem.
- `schemas.py`
  - **What:** Two Pydantic classes: `TicketCategory` (`category` as `Literal["billing", "technical", "general"]`, `reason`, `sure: bool`) and `TicketRecord` (customer, ticket, category, reply, `used_fallback`, `needs_human_review`, `review_reasons`). The three fields added after Step 1 have defaults, so older code still builds a record.
  - **Why:** The classifier's answer is checked, and every ticket's result has the same fields — easy to print, store or review.
- `models/llm.py`
  - **What:** A `MODEL_NAME` constant and one function, `get_model()`, which returns `ChatOpenAI(model=MODEL_NAME, temperature=0)`.
  - **Why:** Every chain and agent gets the same model and settings from here. To try another model you change one line, not every file.
- `chains/classifier.py`
  - **What:** `build_classifier_chain()` returns `prompt | get_model().with_structured_output(TicketCategory)`, wrapped with `with_unsure_category()`; `classify_ticket(ticket)` invokes it and returns the `TicketCategory`.
  - **Why:** Each kind of ticket needs a different reply. Sorting first means each reply prompt stays short and focused.
- `chains/replies.py`
  - **What:** `build_reply_chain(category)`: load the prompt file with the category's name, build `lookups | prompt | model | parser`, and wrap it with `with_canned_reply()`.
  - **Why:** Each category gets its own rules, so billing and technical replies each stay on topic.
- `services/pipeline.py`
  - **What:** `stream_reply(chain, inputs)` prints each chunk from `chain.stream(inputs)` and returns the full text; `review_reasons(category, used_fallback)` returns the list of reasons for a human check; `handle_ticket(ticket, customer_id)` classifies, builds the reply chain, streams the reply, and returns a `TicketRecord`.
  - **Why:** The whole flow is one function that `main.py` and the tests can call, and the reply appears word by word instead of all at once.
- `prompts/classifier.txt`
  - **What:** This text file holds the classifier prompt. The code loads it with `load_prompt("classifier")`.

    - **Why:** You can change the rules for one kind of ticket without touching the code or the other prompts.
- `prompts/billing.txt`
    - **What:** This text file holds the billing prompt. The code loads it with `load_prompt("billing")`.
    - **Why:** You can change the rules for one kind of ticket without touching the code or the other prompts.
- `prompts/technical.txt`
    - **What:** This text file holds the technical prompt. The code loads it with `load_prompt("technical")`.
    - **Why:** You can change the rules for one kind of ticket without touching the code or the other prompts.
- `prompts/general.txt`
    - **What:** This text file holds the general prompt. The code loads it with `load_prompt("general")`.
    - **Why:** You can change the rules for one kind of ticket without touching the code or the other prompts.
- `utils/logger.py`
    - **What:** One function, `get_logger(name)`. It gets the logger for that name and, only the first time (`if not logger.handlers:`), adds a console handler with a timestamp format and the level from `LOG_LEVEL`.
    - **Why:** Every file logs in the same format with one line of code. Without the `if not logger.handlers:` guard, every log line would print twice.
- `utils/prompts.py`
    - **What:** One constant, `PROMPTS_DIR` (the prompts folder, from the repo root), and one function, `load_prompt(name)`, which opens `PROMPTS_DIR + "/" + name + ".txt"` and returns its text.
    - **Why:** Prompts live in text files, so you can change how an agent behaves without editing Python — and no agent has to open files itself.
- `tests/unit/test_schemas.py`
    - **What:** Three test functions, no API key: `test_good_category()` builds a `TicketCategory` with `billing`, `test_unknown_category()` checks that `shipping` raises `ValidationError`, and `test_record()` builds a `TicketRecord` from the 4 Step 1 fields. Each prints its case line.
    - **Why:** It proves the classifier can never return a category no branch handles, and that old records still build after new fields are added.
- `tests/integration/test_live_pipeline.py`
    - **What:** Test functions that run `handle_ticket()` on the sample tickets with the real API, print each category next to the expected one, and check that the unclear ticket is flagged for a human.

    - **Why:** It proves each ticket gets the right category and a real reply.
- `data/sample/tickets.json`
    - **What:** A JSON list of 4 test tickets, each with a `customer_id`, the `ticket` text and the `expected` category.
    - **Why:** The same inputs every run, shared by the app and the tests.

Step 2 — Customer Lookups That Run at the Same Time

Project: LangChainPro-Branching-Parallel-And-Retry-Patterns-For-A-Real-Customer-Support-Pipeline — Step 2 of 4

- **Where you are now:** after Step 1 every ticket gets a category and a reply — but billing and technical replies know nothing about the customer.
- **What's new in this step:** `chains/lookups.py` (two fake, 1-second lookups and a `RunnableParallel` step), `{account}` and `{orders}` in two prompts, a timing script and a test.
- **What stays the same:** the classifier, `handle_ticket()`, `main.py`, and the general reply chain.
- **Why:** the two lookups don't need each other. Running them one after another wastes a full second per ticket, for nothing.
- **How:** `RunnableParallel({"ticket": ..., "account": ..., "orders": ...})` — every function gets the same input, they run at the same time, and the output dict fills the prompt. It goes in front of the billing and technical chains with `|`.
- **Working after this step:** replies use the account data, and `python scripts/time_lookups.py` shows about 2.0 s vs. 1.0 s.

Read first: Doc05 — LCEL — the bullets "a plain Python function becomes a Runnable step" and "a `dict` becomes a step that runs each value in parallel", and the table row "Two independent tasks, same input".

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Add `{account}` and `{orders}` to `prompts/billing.txt` and `prompts/technical.txt`.
2. Write `chains/lookups.py`: `get_ticket_text()`, `get_account_info()`, `get_order_history()` (fake data, `time.sleep(1)` each) and `build_lookup_step()`.
3. Change `build_reply_chain()`: billing and technical start with `build_lookup_step() |`; general stays as it was.
4. Write `scripts/time_lookups.py` and `tests/unit/test_lookups.py`, and run both.

Your files after Step 2:

```
src/langchainpro/
├── chains/
│   ├── lookups.py       (new) the lookups + build_lookup_step()
│   └── replies.py       (changed) lookups before billing/technical
├── prompts/
│   ├── billing.txt      (changed) + {account}, {orders}
│   └── technical.txt    (changed) + {account}, {orders}
└── (Step 1's other files, unchanged)
tests/unit/test_lookups.py  (new) no API key needed
scripts/time_lookups.py     (new) one after another vs. parallel
```

Why these files (what and why):

- `chains/lookups.py`
  - **What:** Fake `FAKE_ACCOUNTS` and `FAKE_ORDERS` data, three small functions that read the inputs (`get_ticket_text`, `get_account_info`, `get_order_history`), and `build_lookup_step()`, which returns a `RunnableParallel` running the lookups at the same time.
  - **Why:** Two lookups that don't depend on each other finish faster together than one after the other.
- `chains/replies.py`
  - **What:** `build_reply_chain(category)`: load the prompt file with the category's name, build `lookups | prompt | model | parser`, and wrap it with `with_canned_reply()`.
  - **Why:** Each category gets its own rules, so billing and technical replies each stay on topic.
- `prompts/billing.txt`
  - **What:** This text file holds the billing prompt. The code loads it with `load_prompt("billing")`.
  - **Why:** You can change the rules for one kind of ticket without touching the code or the other prompts.
- `prompts/technical.txt`
  - **What:** This text file holds the technical prompt. The code loads it with `load_prompt("technical")`.
  - **Why:** You can change the rules for one kind of ticket without touching the code or the other prompts.
- `tests/unit/test_lookups.py`
  - **What:** Three test functions, no API key: a known customer gets account and order data, an unknown customer gets the "not found" texts, and `build_lookup_step()` returns the keys `ticket`, `account`, `orders` in under 1.5 seconds.
  - **Why:** It catches a change that quietly makes the lookups run one after another again.
- `scripts/time_lookups.py`

- **What:** One `main()` that times the two lookups one after the other, then with the parallel step, and prints both times.
- **Why:** It shows you, with real numbers, how much time `RunnableParallel` saves.

Step 3 — Retries, Then a Safe Fallback, When a Model Call Fails

Project: LangChainPro-Branching-Parallel-And-Retry-Patterns-For-A-Real-Customer-Support-Pipeline — Step 3 of 4

- **Where you are now:** after Step 2 the pipeline works on a good day — but one outage, even after the client's retries, crashes the ticket.
- **What's new in this step:** `chains/fallback.py` with a canned reply and an "unsure general" category, both used through `.with_fallbacks()` — only for transient errors — and a test that forces a real outage.
- **What stays the same:** the lookups, the prompts, the reply flow.
- **Why:** real API calls fail at any real volume. The retry part already exists — the OpenAI client retries 2 times with backoff (Doc04) — so adding `.with_retry()` would stack a second layer. What's missing is what happens after the retries run out.
- **How:** `chain.with_fallbacks([RunnableLambda(canned_reply)], exceptions_to_handle=TRANSIENT_ERRORS)` around each reply chain and the classifier. `TicketRecord` gets a `used_fallback` field with a default.
- **Working after this step:** when the model can't be reached, the ticket still gets a safe reply; a bad key or a bug still fails loudly.

Read first: Doc04 — Error types, Doc02 — Retrying, Doc05 — LCEL — the `.with_retry()` / `.with_fallbacks()` bullets and "one retry layer only".

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Write `chains/fallback.py`: `TRANSIENT_ERRORS`, `FALLBACK_REPLY`, `canned_reply()`, `unsure_category()`, `with_canned_reply(chain)`, `with_unsure_category(chain)`. Write your answer to the double-charge question as a comment at the top.
2. Wrap the classifier chain and every reply chain.
3. Add `used_fallback: bool = False` to `TicketRecord`. In `handle_ticket()`, set it (the reply equals `FALLBACK_REPLY`). In `main.py`, drop the `RateLimitError` catch and add one `print_record()` line for `used_fallback`.
4. Write `tests/unit/test_fallback.py`: a real outage falls back, a working chain doesn't, a bug still raises.

Your files after Step 3:

```
src/langchainpro/
├── main.py              (changed) prints used fallback
├── schemas.py           (changed) + used_fallback
├── chains/
│   ├── fallback.py      (new) backups for transient errors only
│   ├── classifier.py    (changed) + with_unsure_category()
│   └── replies.py       (changed) + with_canned_reply()
├── services/
│   └── pipeline.py      (changed) + used_fallback
└── (the other files, unchanged)
tests/unit/test_fallback.py  (new) no API key needed
```

Why these files (what and why):

- `main.py`
  - **What:** `main()` loads the config, reads `tickets.json`, runs `handle_ticket()` for every ticket, and prints each `TicketRecord` with `print_record()`.
  - **Why:** One starting point that runs the ticket queue and prints the results. The chains live in other files.
- `schemas.py`
  - **What:** Two Pydantic classes: `TicketCategory` (`category` as `Literal["billing", "technical", "general"]`, `reason`, `sure: bool`) and `TicketRecord` (customer, ticket, category, reply, `used_fallback`, `needs_human_review`, `review_reasons`). The three fields added after Step 1 have defaults, so older code still builds a record.
  - **Why:** The classifier's answer is checked, and every ticket's result has the same fields — easy to print, store or review.
- `chains/fallback.py`
  - **What:** `TRANSIENT_ERRORS` (connection errors and timeouts, rate limits, server errors), a `FALLBACK_REPLY` text, `canned_reply()` and `unsure_category()` fallback functions, and `with_canned_reply(chain)` / `with_unsure_category(chain)`, which call `chain.with_fallbacks([...], exceptions_to_handle=TRANSIENT_ERRORS)`.
  - **Why:** During an outage the customer still gets a polite reply, while a bad key or a code bug still fails loudly.
- `chains/classifier.py`
  - **What:** `build_classifier_chain()` returns `prompt | get_model().with_structured_output(TicketCategory)`, wrapped with `with_unsure_category()`; `classify_ticket(ticket)` invokes it and returns the `TicketCategory`.
  - **Why:** Each kind of ticket needs a different reply. Sorting first means each reply prompt stays short and focused.
- `chains/replies.py`

  - **What:** `build_reply_chain(category)` : load the prompt file with the category's name, build `lookups | prompt | model | parser`, and wrap it with `with_canned_reply()`.
  - **Why:** Each category gets its own rules, so billing and technical replies each stay on topic.
- `services/pipeline.py`
  - **What:** `stream_reply(chain, inputs)` prints each chunk from `chain.stream(inputs)` and returns the full text; `review_reasons(category, used_fallback)` returns the list of reasons for a human check; `handle_ticket(ticket, customer_id)` classifies, builds the reply chain, streams the reply, and returns a `TicketRecord`.
  - **Why:** The whole flow is one function that `main.py` and the tests can call, and the reply appears word by word instead of all at once.
- `tests/unit/test_fallback.py`
  - **What:** A `ChatOpenAI` pointed at a port where nothing listens, plus a working step and a buggy step. Three test functions check that a real outage gives `FALLBACK_REPLY`, a working chain never uses it, and a `KeyError` is still raised.
  - **Why:** It proves the fallback fires during an outage and never hides a bug — with no key and no cost.

Step 4 — Stream the Reply Live and Build One Structured Record per Ticket

Project: LangChainPro-Branching-Parallel-And-Retry-Patterns-For-A-Real-Customer-Support-Pipeline — Step 4 of 4

- **Where you are now:** after Step 3 the pipeline survives failures — but the reply appears all at once, and the unclear ticket and every fallback pass without a flag.
- **What's new in this step:** `stream_reply()`, `review_reasons()`, two new `TicketRecord` fields (`needs_human_review`, `review_reasons`), two new `print_record()` lines, and `tests/unit/test_review.py`.
- **What stays the same:** the classifier, the lookups, the fallbacks, and the names and return types of `handle_ticket()` and `print_record()`.
- **Why:** one reply is needed in two shapes — live text for the person watching, a checked record for the ticketing system — and both must come from the same reply.
- **How:** `chain.stream(inputs)` → print each piece with `flush=True` and join them → `review_reasons()` → the two new fields in `TicketRecord(...)`.
- **Working after this step:** the full project — each reply streams live, and each ticket ends with a record that says whether a human should review it, and why.

Read first: Doc05 — LCEL ("every chain is a `Runnable` : `.invoke()`, `.stream()`…"), Doc04 — Streaming, Doc04 — Structured output.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Add `needs_human_review: bool = False` and `review_reasons: list[str] = []` to `TicketRecord`.
2. In `services/pipeline.py`, write `stream_reply(chain, inputs)` and `review_reasons(category, used_fallback)`. In `handle_ticket()`, swap `.invoke()` + print for `stream_reply()`, and fill the two new fields.
3. In `print_record()`, print the review flag and each reason.
4. Write `tests/unit/test_review.py` and add a flag check to the integration test. `test_schemas.py` stays unchanged. Run everything — ticket 4 should be flagged.

Your files after Step 4 (final): the full tree in Project Structure. What changed:

```
src/langchainpro/
├── main.py              (changed) print_record() + review lines
├── schemas.py           (changed) + needs_human_review, review_reasons
└── services/
    └── pipeline.py      (changed) + stream_reply(), review_reasons()
tests/
├── unit/test_review.py               (new) no API key needed
└── integration/test_live_pipeline.py (changed) + review flags
```

Why these files (what and why):

- `main.py`
  - **What:** `main()` loads the config, reads `tickets.json`, runs `handle_ticket()` for every ticket, and prints each `TicketRecord` with `print_record()`.
  - **Why:** One starting point that runs the ticket queue and prints the results. The chains live in other files.
- `schemas.py`
  - **What:** Two Pydantic classes: `TicketCategory` (`category` as `Literal["billing", "technical", "general"]`, `reason`, `sure: bool`) and `TicketRecord` (customer, ticket, category, reply, `used_fallback`, `needs_human_review`, `review_reasons`). The three fields added after Step 1 have defaults, so older code still builds a record.
  - **Why:** The classifier's answer is checked, and every ticket's result has the same fields — easy to print, store or review.
- `services/pipeline.py`
  - **What:** `stream_reply(chain, inputs)` prints each chunk from `chain.stream(inputs)` and returns the full text; `review_reasons(category, used_fallback)` returns the list of reasons for a human check; `handle_ticket(ticket, customer_id)` classifies, builds the reply chain, streams the reply, and returns a `TicketRecord`.
  - **Why:** The whole flow is one function that `main.py` and the tests can call, and the reply appears word by word instead of all at once.
- `tests/unit/test_review.py`

    - **What:** A helper `make_category(sure)` and three test functions that call `review_reasons()`: a sure ticket gets no reasons, an unsure ticket gets one, and a fallback reply gets one.
    - **Why:** Each review rule is checked alone, so a rule that stops firing is noticed at once.
- `tests/integration/test_live_pipeline.py`
    - **What:** Test functions that run `handle_ticket()` on the sample tickets with the real API, print each category next to the expected one, and check that the unclear ticket is flagged for a human.
    - **Why:** It proves each ticket gets the right category and a real reply.

Final Deliverable: LangChainPro-Branching-Parallel-And-Retry-Patterns-For-A-Real-Customer-Support-Pipeline — a support-ticket pipeline in a proper package layout, built only from LCEL: a structured classifier, a reply chain per category, parallel lookups, fallbacks on transient errors only, a live-streamed reply, and a checked record per ticket.

Why this is not an agent (said plainly): nothing here decides what to do next at run time, and nothing loops. The path is fixed: classify, reply, record. That's the point — it's the LangChain side of the line that Project 3 crosses.

Run, Test and Configure the Final Project

The same commands and settings as the repo's README.md — once Step 4 is done, this is how the finished project is run, tested and configured.

Configuration

Variable	Required	Default	Meaning
`OPENAI_API_KEY`	yes	—	Your OpenAI API key
`LOG_LEVEL`	no	`INFO`	`DEBUG`, `INFO`, `WARNING` or `ERROR`

Other settings live in code, each in one place:

Setting	Where	Default
Chat model	`models/llm.py` → `MODEL_NAME`	`gpt-4o-mini`
Errors that fall back	`chains/fallback.py` → `TRANSIENT_ERRORS`	connection, rate limit, 5xx
The canned reply	`chains/fallback.py` → `FALLBACK_REPLY`	"a member of our team will reply…"

Setting	Where	Default
Fake lookup delay	`chains/lookups.py` → `LOOKUP_DELAY_SECONDS`	`1`
The ticket queue	`main.py` → `SAMPLES_PATH`	`data/sample/tickets.json`

Your own tickets: edit `data/sample/tickets.json` — each ticket needs a `customer_id`, a `ticket` and an `expected` category.

Usage

Run from the repo root:

```
python -m langchainpro.main     # or simply: langchainpro
```

To see the parallel lookups save time (no API key needed):

```
python scripts/time_lookups.py
```

Running the Tests

Run from the repo root:

```
# unit tests — no API key
python tests/unit/test_schemas.py
python tests/unit/test_lookups.py
python tests/unit/test_fallback.py
python tests/unit/test_review.py

# integration tests — real API calls, needs OPENAI_API_KEY
python tests/integration/test_live_pipeline.py
```

`test_fallback.py` forces a real outage (a model pointed at a local port where nothing listens) and checks the fallback fires — and that a working chain and a real bug don't trigger it.

Checklist Before You Call This Done

- [ ] The repo folder name matches the GitHub repo name exactly
- [ ] Code lives in `src/langchainpro/`, installed with `pip install -e .`; runs with `python -m langchainpro.main`

- [ ] Every folder has one clear job, and you can say why it exists
- [ ] Only LCEL — no agent loop, no LangGraph — and you can say why none is needed
- [ ] The classifier returns a checked `TicketCategory`; every decision is logged with its reason
- [ ] `time_lookups.py` shows the parallel step is really faster
- [ ] Only transient errors fall back; a bad key and a bug still fail loudly — all three proven by `test_fallback.py`
- [ ] One retry layer only (the client's), and you can explain why there's no `.with_retry()`
- [ ] Replies stream live, and the record is built from that same reply
- [ ] The unclear ticket and every fallback are flagged for a human, with the reason
- [ ] `tests/unit/` pass with no API key; `tests/integration/` pass with one
- [ ] `.env` is not in git
- [ ] You can explain every line of your own code, without help

Publish It to GitHub

1. Copy README.md from this folder into your repo folder, and put your own name in its License line.
2. Check nothing secret is staged, then commit:

```
git status                      # .env and .venv/ must NOT appear
git add .
git commit -m "LangChainPro: an LCEL-only support-ticket pipeline"
```

3. On GitHub, create a new repo named exactly `LangChainPro-Branching-Parallel-And-Retry-Patterns-For-A-Real-Customer-Support-Pipeline` (no README — you have one). In the repo's **Description** box, paste the **Short description** line from the top of README.md. Then:

```
git branch -M main
git remote add origin <your-repo-url>
git push -u origin main
```

Status

Not started. Track your progress in ../PROGRESS.md.

Stuck? Ask for **Hint 1** or **Hint 2** about the exact part you're stuck on. Only ask for the full code if you say **"Show me the solution."**